

Property Graphs: Laboratory 1 Report

Davide Volpi

Jan Mantsch

davide.volpi@estudiantat.upc.edu

jan.christian.mantsch@estudiantat.upc.edu

1 A1

The proposed property graph schema was designed to store research publications while optimizing for data integrity, graph traversal efficiency, and strict adherence to the domain constraints. The comprehensive evolution of this schema—from the baseline (Part A1), through the peer-review expansion (Part A3), to the final recommender system layout (Part C), is visually mapped out in **Figure 1**. Below is a description of the baseline schema (**Black** elements in the diagram) and the justification for our foundational design decisions.

Graph Schema (Nodes, Properties, and Relationships)

The schema contains several entities and relationships derived from the domain:

- **Nodes:**
 - **Paper:** Represents the core entity, containing attributes such as ID/DOI, Title, Year, Pages, and an abstract.
 - **Author:** Represents individuals who write research papers. Contains ID, Name, and Email.
 - **Keyword:** Represents topics related to the papers, containing a Name property.
 - **Venues & Groupings:** Separated into **Conference**, **Workshop**, and **Journal**. Intermediate grouping nodes include **Proceedings** (for conferences/workshops) and **Journal Volume** (for journals).
 - **City:** Represents the venue location, containing a Name.
- **Edges:** Relationships capture domain interactions, such as **writes**, **reviews**, **cites**, **about**, **published-in**, **part-of**, and **is-in**.

Design Justifications

We made modelling choices to satisfy the requirements of maintenance, reusability, non-redundancy, and query performance:

- **Non-redundancy (Edge Properties):** A paper can be written by many authors, however only one of them acts as corresponding author. Instead of creating separate relationship types (e.g., **WRITES** vs **IS_CORRESPONDING**), the model uses a single **writes** relationship with a boolean property **{is_corr}**. This prevents big relationship and keeps the graph structurally clean.
- **Reusability (Author Nodes as Reviewers):** Reviewers are scientists and therefore they are relevant authors. Rather than creating a separate **Reviewer** node type, our schema reuses the **Author** node, applying a **reviews** relationship to the **Paper**. This ensures that any updates to a scientist’s profile only happen in one place. Furthermore, the author of a certain paper cannot be reviewer of her own paper.
- **Maintenance (Hierarchical Venue Modelling):** The publication paths are normalized to enforce that each paper can only be published in a single conference, workshop or journal. Because a conference/workshop is organized in terms of editions and proceedings are published records which include all the papers presented in an edition, we address conference papers through a **Proceedings** node. Conversely, a journal publishes accepted

papers in terms of volumes, so journal papers route through a **Journal Volume** node. This hierarchy eliminates update anomalies if a venue changes its name.

- **Query Performance (Nodes vs. Properties):** Entities frequently used for aggregations, such as **Keyword** and **City**, are modelled as distinct nodes rather than string properties. We extract **City** as a standalone node because each edition of a conference/workshop is held in a given city, and a paper can be about one or more topics, specified by means of keywords.

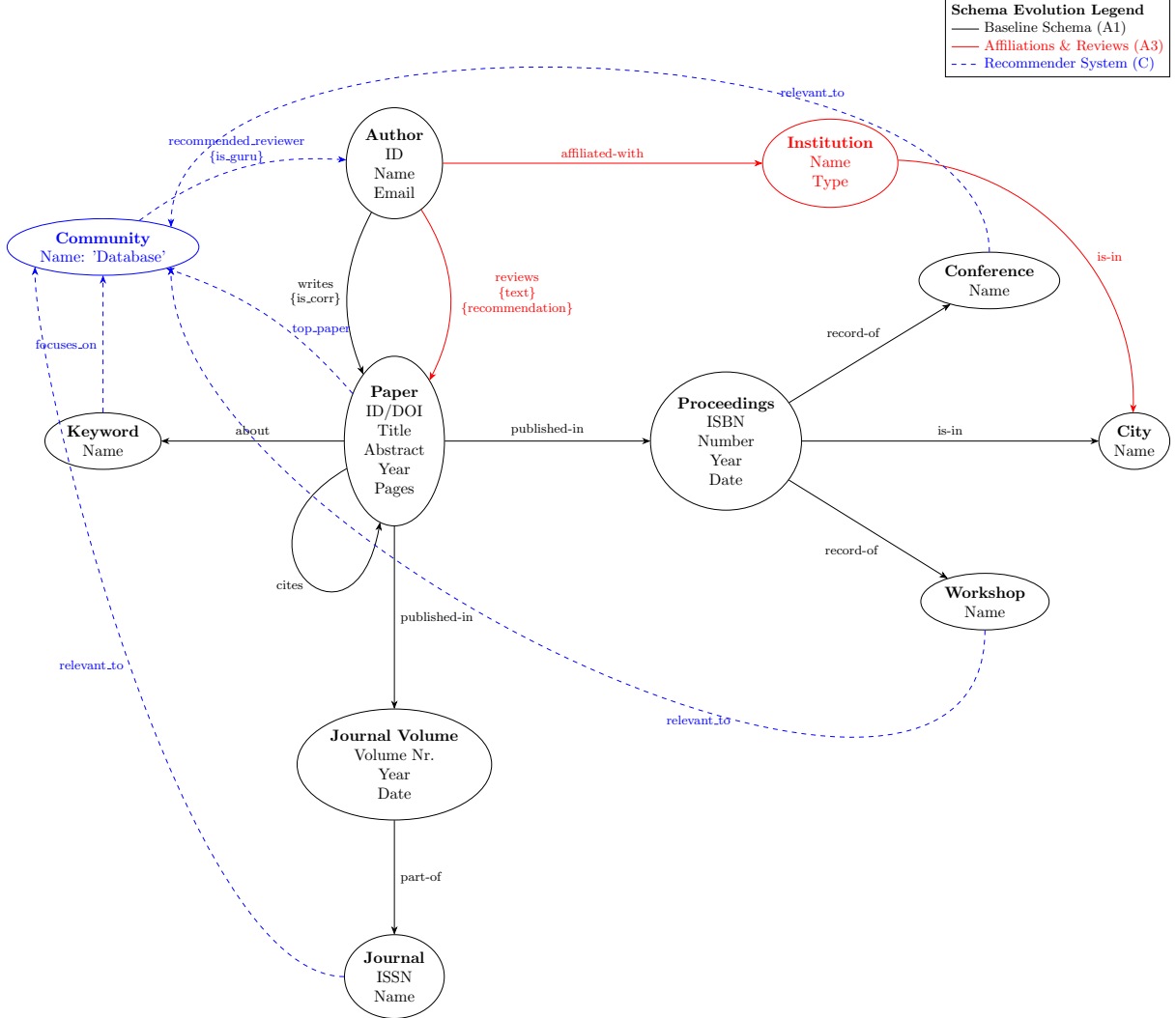


Figure 1: Comprehensive Graph Schema encompassing baseline features and subsequent structural evolutions.

2 A2

To populate our graph database, we developed a automated Python pipeline. Because real-world academic datasets do not contain all the specific concepts, relationships, and edge-case scenarios required by the domain model and the subsequent queries in this lab, our approach relies on a hybrid of real data extraction and synthetic data generation.

The pipeline processes the data using **pandas** and exports it to CSV files, which are then loaded into Neo4j using the **neo4j** Python driver and **LOAD CSV** Cypher queries.

Real Data Extraction and Pre-processing

The basic data was fetched using the Semantic Scholar REST API. To ensure a dense citation network (which is crucial for calculating the Impact Factor in Section B), we constrained our API requests to fetch papers within a strict, continuous time interval.

- **Data Retrieval:** We utilized the `/paper/search/bulk` endpoint to retrieve batches of `paperIds`, followed by the `/paper/batch` endpoint to fetch detailed metadata (title, year, abstract, authors, publication venue).
- **Pre-processing:** The raw JSON responses were parsed and flattened into relational-style DataFrames (`paper_details_df`, `authors_df`, `publications_df`). Nested arrays, such as the lists of authors for a single paper, were exploded into separate rows to facilitate the creation of the `writes` relationships. Missing values for crucial identifiers were dropped to maintain graph integrity.

Synthetic Data Generation

To full fill the schema requirements and prepare the graph for the specific queries in Sections A.3, B, and C, we generated synthetic data using Python’s `numpy` and `random` libraries.

Basic Node Properties & Relationships:

- **Citations:** We simulated the `cites` network by assigning each paper between 1 and 10 random references sampled from the existing pool of fetched `paperIds`.
- **Venues:** Proceedings were randomly distributed with an 80% probability of being a `Conference` and a 20% probability of being a `Workshop`. We assigned random host cities and generated fake 9-digit ISBNs.
- **Paper & Author details:** We generated random page ranges for papers and created synthetic email addresses for authors based on their names.

Preparation for Graph Evolution (Section A.3):

- **Reviews:** The domain specifies that papers have reviewers and that authors cannot review their own papers. Our script enforces this by subtracting a paper’s actual author IDs from the total author pool before selecting up to 3 random reviewers. We also programmatically generated a random 50-character string to serve as the review `text` and assigned a random `recommendation` (`accepted` or `rejected`).
- **Affiliations:** We mapped authors to synthetic institutions categorized strictly by `Type` as either a `University` (e.g., "University of Barcelona") or a `Company` (e.g., "Meta Tokyo", "Nvidia Rome").

Preparation for Recommender (Section C):

- **Keywords & Communities:** Section C requires identifying venues specific to the database community, defined by having $\geq 90\%$ of their papers containing database keywords. To guarantee this, our script first assigns a "dominant community" to each venue. Then, when assigning 4 keywords to a paper, there is a 95% probability that 3 of those keywords will be drawn strictly from the venue’s dominant community (e.g., "data management", "indexing"), with a 5% chance of drawing from a random outlier community. This safely clears the 90% threshold required for the Cypher queries.

Enforcing Lab Guarantees

If we relied purely on random distribution, the probability of satisfying the complex edge cases required by the lab queries would be near zero. To guarantee that our Cypher queries retrieve at

least one instance of data, we implemented an `enforce_lab_guarantees` function to forcefully inject specific conditions before saving the CSVs:

- **Journal Volumes (Query B.3):** Beyond just forcing a `pub.vol` (Volume Number), the script precisely aligns publication dates to satisfy the temporal constraints of the Impact Factor formula. It forces two journal papers to be published in 2021 and forces eight other papers published in 2022 to cite them, guaranteeing a valid calculation window.
- **The Loyal Author (Query B.2):** To ensure at least one author has published in 4 *different editions* of the same venue, the script isolates a conference, generates four distinct annual proceedings for it (e.g., 2018, 2019, 2020, 2021), and appends a single, specific `authorId` to one paper in each edition.
- **The Database Guru (Query C.4):** To guarantee the existence of an author with at least two papers in the top-100 most cited database papers, the script selects two database papers, assigns a specific author to both, and then forces 50 other database papers to cite these two specific papers. This injection of citations guarantees they reach the top 100.

Loading the Data

After the DataFrames are fully generated, they are exported as `papers.csv`, `authors.csv`, `publications.csv`, `reviews.csv`, and `affiliations.csv`. A Python script then reads a Cypher template file, resolves the absolute paths to these CSVs, and executes the `LOAD CSV` statements against the Neo4j instance to construct the nodes and relationships.

3 A3

As the domain requirements evolved to track detailed peer-review data and author affiliations, we extended our baseline schema. The updated elements are highlighted in **Red** in Figure 1. Below, we highlight the changes and justify our design choices.

Modifications to the Schema

The following changes were introduced to the original graph model:

- **New Nodes:** We introduced the `Institution` node to represent where an author works. It contains the properties `Name` and `Type` (indicating whether it is a University or a Company).
- **New Relationships:**
 - `(Author)-[:affiliated-with]->(Institution)`: Connects an author to their workplace.
 - `(Institution)-[:is-in]->(City)`: Connects the workplace to its physical location, reusing the existing `City` nodes.
- **Updated Relationships:** The existing `reviews` relationship between `Author` and `Paper` was augmented with two new properties: `text` (the written content of the review) and `recommendation` (the suggested decision: accepted or rejected).

Design Justifications

The decisions for these updates were made to ensure continued query performance, non-redundancy, and reusability:

- **Query Performance (Nodes vs. Properties for Affiliations):** Instead of adding "Institution" and "City" as string properties directly on the `Author` node, we modelled `Institution` as an explicit node, to optimize graph traversal. If a query asks to "find all authors working in Tokyo", the database simply finds the single `City` node for Tokyo

and traverses the incoming `is-in` and `affiliated-with` edges. This index lookup and traversal ($O(1)$) is more efficient than scanning every single author node ($O(N)$) to check string properties.

- **Simplicity & Maintenance (Node Property for Institution Type):** Universities and Companies behave identically within the structure of our graph (authors work there, and they are located in cities). Therefore, instead of creating separate `University` and `Company` nodes (which would duplicate relationship logic), we use a single `Institution` node with a `Type` property, to keep the schema concise while allowing easy filtering.
- **Reusability (Reusing City Nodes):** By connecting the new `Institution` nodes to the existing `City` nodes, we reuse existing entities. This integrates the new affiliation data with the existing conference/workshop location data, enabling cross-domain queries (e.g., finding authors who published a paper at a conference hosted in the same city as their university).
- **Non-redundancy (Edge Properties for Reviews):** A review represents an interaction between a specific reviewer (Author) and a specific Paper. Rather than creating a new intermediate `Review` node, which would require two new relationships (`WROTE_REVIEW`, `REVIEW_OF`) and bloat the graph, we placed the `text` and `recommendation` attributes directly onto the `reviews` edge. This models the interaction as a rich relationship while maintaining structural simplicity.

4 B

B.1 Top 3 most cited papers of each conference/workshop

```

1 MATCH (p:Paper)-[:'published-in']->(proc:Proceedings)-[:'record-of']->(venue)
2 WHERE venue:Conference OR venue:Workshop
3 OPTIONAL MATCH (citing_paper:Paper)-[:cites]->(p)
4 WITH venue, p, COUNT(citing_paper) AS citations
5 ORDER BY coalesce(venue.conferenceId, venue.workshopId), citations DESC
6 WITH venue, COLLECT(p)[0..3] AS top_papers
7 RETURN coalesce(venue.conferenceId, venue.workshopId) AS venue, top_papers;
```

Modelling Motivation: We can target both venue types in a single pattern match without needing a complex UNION. We utilize an `OPTIONAL MATCH` for the incoming `cites` edges to ensure that papers with zero citations are not dropped from the dataset. Grouping by venue and ordering by citations descending allows us to use Cypher’s list slicing on the aggregated collection to isolate the top 3 papers per venue.

B.2 Community of each conference/workshop

```

1 MATCH (a:Author)-[:writes]->(p:Paper)-[:'published-in']->(proc:Proceedings)-[:'
   record-of']->(venue)
2 WHERE venue:Conference OR venue:Workshop
3 WITH venue, a, COUNT(DISTINCT proc) AS distinct_editions
4 WHERE distinct_editions >= 4
5 RETURN coalesce(venue.conferenceId, venue.workshopId) AS venue, COLLECT(a.name)
   AS community;
```

Modelling Motivation: Because our schema models the “Edition” of a venue as a `Proceedings` node, we can easily verify how many distinct editions an author has participated in by counting the distinct `Proceedings` nodes traversed. The `COUNT(DISTINCT proc)` is important here: an author might publish multiple papers in the *same* edition, but the query strictly requires participation across 4 *different* editions to qualify as a community member.

B.3 Impact factor of journals

```
1 MATCH (j:Journal)-[:'part-of']-(v:JournalVolume)-[:'published-in']-(p:Paper)
2 WITH j, p, v, 2022 AS target_year
3 WHERE v.year = target_year - 1 OR v.year = target_year - 2
4 WITH j, target_year, COUNT(p) AS citable_items, COLLECT(p) AS prev_papers
5 WHERE citable_items > 0
6 UNWIND prev_papers AS target_paper
7 OPTIONAL MATCH (citing_paper:Paper)-[:cites]->(target_paper)
8 WHERE citing_paper.publicationDate.year = target_year
9 WITH j, citable_items, COUNT(citing_paper) AS citations
10 RETURN j.name AS Journal, toFloat(citations)/citable_items AS ImpactFactor ORDER
    BY ImpactFactor DESC;
```

Modelling Motivation: Our schema separates Journals from Conferences, meaning this query can bypass all conference proceedings and go directly into the JournalVolume hierarchy. We calculate the denominator (`citable_items`) first by matching papers to volumes published in $Y - 1$ or $Y - 2$. We then UNWIND those target papers and execute an OPTIONAL MATCH to find incoming citations strictly from the target year. The division uses `toFloat()` to prevent integer division truncation.

B.4 h-index of the authors

```
1 MATCH (a:Author)-[:writes]->(p:Paper)
2 OPTIONAL MATCH (p)-[:cites]-(citing_paper:Paper)
3 WITH a, p, count(citing_paper) AS citations
4 ORDER BY citations DESC
5 WITH a, collect(citations) AS citationList
6 RETURN a.name AS Author, a.authorId AS AuthorID, size(citationList) AS
    TotalPapers, citationList AS CitationArray, size([i IN range(1, size(
    citationList)) WHERE citationList[i-1] >= i]) AS h_index ORDER BY h_index
    DESC, TotalPapers DESC;
```

Modelling Motivation: By sorting an author's citation counts in descending order and aggregating them into a `citationList`, we can use a Cypher list comprehension to generate an index range (1 to `TotalPapers`). The WHERE clause filters this array to only keep papers where the citation count is greater than or equal to its rank position. The size of this resulting list is, by definition, the author's h-index.

5 C

The structural elements introduced in this step are highlighted in **dashed blue** in Figure 1.

C.1 Create Database Community Node and relationships with Keywords

```
1 CREATE (c:Community {name: 'Database'})
2 WITH c
3 MATCH (k:Keyword)
4 WHERE k.name IN ['data management', 'indexing', 'data modeling', 'big data', '
    data processing', 'data storage', 'data querying']
5 MERGE (c)-[:focuses_on]->(k);
```

Modelling Motivation: We model the community as a standalone node (`c:Community`) rather than applying labels directly to millions of papers. By creating `[:focuses_on]` relationships connecting the central community to specific keywords, we avoid a massive bottleneck. The community logically links to papers via the keywords they are `[:about]`. We utilize MERGE on the edge to ensure query idempotency.

C.2 Find venues relevant to the Database community

```
1 MATCH (v:Workshop|Conference|Journal)-[:'record-of'|'part-of']-(c)-[:'published
    -in']-(dp:Paper)-[:about]->(k:Keyword)-[:focuses_on]-(c:Community {name: '
    Database'})
```

```

2 WITH c, v, COUNT(DISTINCT dp) AS database_papers
3 MATCH (v)-[:'record-of'|'part-of']-(c)-[:'published-in']-(tp:Paper)
4 WITH c, v, database_papers, COUNT(DISTINCT tp) AS total_papers
5 WHERE database_papers >= 0.9 * total_papers
6 MERGE (v)-[:'relevant_to']->(c);

```

Modelling Motivation: To find venues with a high concentration of database papers, we leverage Neo4j’s multi-label matching feature (`v:Workshop|Conference|Journal`) to traverse all venue types simultaneously. The `WITH` clause acts as an implicit `GROUP BY`, allowing us to calculate the database-specific papers against the total papers. We use the threshold equation using a float (0.9) so Neo4j handles the condition safely without truncating integer division. We then assert the `[:relevant_to]` edge to mark passing venues.

C.3 Find top 100 most cited papers relevant to the Database community

```

1 MATCH (p:Paper)-[:'published-in']->(c)-[:'record-of'|'part-of']->(v:Workshop|
   Conference|Journal)-[:'relevant_to']->(c:Community {name: 'Database'})
2 WITH p, c
3 MATCH (c)-[:'focuses_on']->(k:Keyword)-[:'about']-(op:Paper)-[:'cites']->(p)
4 WITH p, c, COUNT(DISTINCT op) AS citations
5 ORDER BY citations DESC LIMIT 100
6 MERGE (c)-[:'top_paper']->(p);

```

Modelling Motivation: This query is highly optimized because it uses the changes made in step C.2. Rather than relying on keywords to identify candidates, it traverses backward: `Paper -> Venue -> Community`. This guarantees that candidate papers (`p`) are strictly published within established Database Venues. The second `MATCH` ensures that incoming citations are only counted if they come from other papers (`op`) that also map back to Database keywords. The top 100 winners are saved via the `[:top_paper]` edge.

C.4 Find authors of top papers and create recommended reviewer relationships

```

1 MATCH (a:Author)-[:writes]->(p:Paper)-[:'top_paper']-(c:Community {name: '
   Database'})
2 WITH a, c, COUNT(DISTINCT p) AS top_papers_count
3 MERGE (c)-[:'recommended_reviewer']->(a) ON CREATE SET e.is_guru = (
   top_papers_count >= 2);

```

Modelling Motivation: Because the `[:top_paper]` edge was explicitly asserted in the previous step, finding the authors of those top papers is now an instantaneous $O(1)$ traversal (`Author -> Paper -> Community`). Instead of creating two structurally separate relationship types for standard reviewers vs. gurus, we consolidate them into a single `[:recommended_reviewer]` edge. By using Cypher’s ability to evaluate boolean conditions dynamically via `SET e.is_guru = (top_papers_count >= 2)`, we seamlessly tag gurus via a boolean property flag.

6 D

To utilize the Neo4j Graph Data Science Library, we first project the required in-memory graphs. For D.1, we project a bipartite graph of papers and keywords. For D.2, we project a directed monopartite graph of paper citations.

```

1 CALL gds.graph.project('paperKeywordGraph', ['Paper', 'Keyword'], 'about');
2 CALL gds.graph.project('citation_graph', 'Paper', 'cites');

```

D.1 Thematic Clustering via Node Similarity

Our first application explores the conceptual overlap between research publications. We utilized the Node Similarity algorithm, applying the Jaccard similarity metric, to identify pairs of papers that share a significant number of keywords.

```

1 CALL gds.nodeSimilarity.stream('paperKeywordGraph',{similarityMetric: 'JACCARD'
2   })
3 YIELD node1, node2, similarity
4 WITH gds.util.asNode(node1) AS Paper1, gds.util.asNode(node2) AS Paper2,
5   similarity
6 MATCH (p1:Paper {paperId: Paper1.paperId})-[:about]->(k:Keyword)<-[:about]-(p2:
7   Paper {paperId: Paper2.paperId})
8 RETURN Paper1.title AS Paper1, Paper2.title AS Paper2, similarity, COLLECT(
9   DISTINCT k.name) AS SharedKeywords ORDER BY similarity DESC;

```

Modelling Motivation & Domain Interpretation: By executing the Node Similarity algorithm over the bipartite paper-keyword network, we calculate the Jaccard index between pairs of papers. This mathematical metric expresses the ratio of the size of the intersection (shared keywords) to the size of the union (all unique keywords applied to both papers combined).

From a domain perspective, the data obtained provides a ranked assessment of how conceptually similar two papers are. A similarity score of 1.0 indicates that they share the exact same set of keywords. This approach is highly effective for discovering thematic clusters of papers beyond individual keyword searches. It allows a recommender system or a researcher to map out highly related literature that discusses identical concepts, even if those papers belong to different venues or do not directly cite one another.

D.2 Citation Lineage via Dijkstra's Shortest Path

Our second application traces the path of knowledge between research publications. We utilized Dijkstra's Shortest Path algorithm to discover the shortest chain of citations linking a newer paper back to an older, foundational paper.

```

1 MATCH path = (source:Paper)-[:cites*3..5]->(target:Paper)
2 WITH source, target LIMIT 1
3 CALL gds.shortestPath.dijkstra.stream('citation_graph', {sourceNode: source,
4   targetNodes: target})
5 YIELD index, sourceNode, targetNode, totalCost, nodeIds
6 RETURN index, gds.util.asNode(sourceNode).title AS SourcePaper, gds.util.asNode(
7   targetNode).title AS TargetPaper, totalCost AS Hops, [nodeId IN nodeIds |
8   gds.util.asNode(nodeId).title] AS CitationChain ORDER BY index;

```

Modelling Motivation & Domain Interpretation: By executing Dijkstra's algorithm over the `cites` network, we determine the minimum number of intermediary papers (represented by `totalCost` or "Hops") required to connect a new concept to an old one. Because our `cites` relationships are unweighted, Dijkstra's effectively behaves as a Breadth-First Search (BFS).

From a domain perspective, the data obtained (the `CitationChain`) represents the literal evolution of a specific scientific idea. If a researcher wants to understand how a modern methodology was derived from a 20-year-old theorem, this query extracts the exact sequence of literature they must read to bridge that gap.